Chapitre 0 — Avant tout | Partie B — Le temps | leçon 0.12 sur 153 | difficulté 1 / 10

0.12. Une lettre toutes les secondes

Écrire DANS la boucle de jeu, et compter les images pour mesurer le temps.

1. Ce qu'il faut comprendre

Jusqu'ici, on écrivait **avant** la boucle de jeu, une fois pour toutes. Ici, on écrit **dedans** : à chaque image, la case (0, 0) reçoit la lettre du moment.

La console ne connaît ni les secondes ni les millisecondes : elle compte les images. Il y en a **60 par seconde** ; une image dure donc 1000 ÷ 60 ≈ **16,7 millisecondes**. `image()` attend l'image suivante : chaque tour de boucle dure une image.

`IMAGES_PAR_SECONDE` et `SECONDES` donnent un nom à ces nombres. `SECONDES * IMAGES_PAR_SECONDE` vaut 1 × 60 = 60 : quand `images` atteint 60, une seconde est passée. Pour changer la vitesse, on ne touche qu'à `SECONDES`.

Limite : `images` est un `uint8_t`, qui ne dépasse pas 255. On compte donc au plus 255 images, environ **4 secondes** : `SECONDES = 5` donnerait 300, trop grand. Le chapitre suivant montre `attendre()`, qui n'a pas cette limite.

Chaque nouvelle lettre **écrase** la précédente, comme au 0.3. Après Z, `lettre` revient à 0 : on repart de A.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **constante (const)** — Une valeur qui a un nom et ne change jamais : « const uint8_t VIES = 3; ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de calcul** — + plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`const uint8_t IMAGES_PAR_SECONDE = 60;  // la console affiche 60 images par seconde`	Une constante : le nom IMAGES_PAR_SECONDE veut dire 60, pour tout le programme. Elle ne changera jamais ; le nom rend le code plus facile à lire. (Le commentaire le dit : « la console affiche 60 images par seconde ».)
`const uint8_t SECONDES = 1;  // le temps entre deux lettres, en secondes`	Une constante : le nom SECONDES veut dire 1, pour tout le programme. Elle ne changera jamais ; le nom rend le code plus facile à lire. (Le commentaire le dit : « le temps entre deux lettres, en secondes ».)
`uint8_t lettre = 0;  // l'indice de la lettre affichée`	Une variable : une boîte appelée lettre, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « l'indice de la lettre affichée ».)
`uint8_t images = 0;  // les images comptées depuis la dernière lettre`	Une variable : une boîte appelée images, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « les images comptées depuis la dernière lettre ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();  // attend l'image suivante : environ 16,7 ms`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « attend l'image suivante : environ 16,7 ms ».)
`images++;  // une image de plus`	images augmente de 1. (Le commentaire le dit : « une image de plus ».)
`if (images == SECONDES * IMAGES_PAR_SECONDE) { // 1 × 60 = 60 images = 1 seconde`	Une décision : si images est égal à SECONDES * IMAGES_PAR_SECONDE, on fait ce qui est entre les accolades. Sinon, on passe. (Le commentaire le dit : « 1 × 60 = 60 images = 1 seconde ».)
`images = 0;  // on recommence à compter`	images prend une nouvelle valeur : 0. (Le commentaire le dit : « on recommence à compter ».)
`lettre++;  // la lettre suivante`	lettre augmente de 1. (Le commentaire le dit : « la lettre suivante ».)
`if (lettre == sizeof(ALPHABET)) lettre = 0;  // après Z, on revient à A`	Une décision : si lettre est égal à sizeof(ALPHABET) alors on fait « lettre = 0 ». Sinon, on passe. (Le commentaire le dit : « après Z, on revient à A ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`poser(0, 0, ALPHABET[lettre]);  // la même case : la lettre écrase la précédente`	Appel de la fonction poser de la console, avec les arguments 0, 0, ALPHABET[lettre]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans

	le fond (colonne et ligne en clair). (Le commentaire le dit : « la même case : la lettre écrase la précédente ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **892 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        55 o  ← la console, à l'allumage  — la mise en route de la console
  main()           49 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard,
EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 1    12 o sur place, dans main()
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <poser>      // pose une tuile sur une case du fond
```

2. UNE CONSTANTE — IMAGES_PAR_SECONDE

```
const uint8_t IMAGES_PAR_SECONDE = 60;   // la console affiche 60 images par seconde
```

3. UNE CONSTANTE — SECONDES

```
const uint8_t SECONDES = 1;              // le temps entre deux lettres, en secondes
```

4. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0;   // l'indice de la lettre affichée
```

5. UNE VARIABLE — IMAGES

```
uint8_t images = 0;   // les images comptées depuis la dernière lettre
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();          // attend l'image suivante : environ 16,7 ms
    images++;         // une image de plus

    if (images == SECONDES * IMAGES_PAR_SECONDE) {  // 1 × 60 = 60 images = 1 seconde
      images = 0;                                   // on recommence à compter
      lettre++;                                     // la lettre suivante
      if (lettre == sizeof(ALPHABET)) lettre = 0;   // après Z, on revient à A
    }

    poser(0, 0, ALPHABET[lettre]);   // la même case : la lettre écrase la précédente
  }
}
```

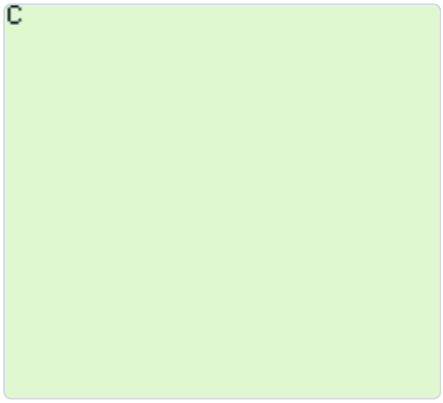
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

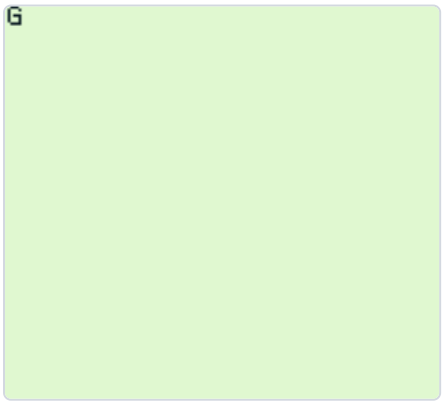
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> En haut à gauche : A, puis B une seconde plus tard, puis C…

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ A au départ

✓ B une seconde plus tard

✓ C la seconde d'après

Ensuite : **0.12.1. Une lettre toutes les secondes — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++